

**ENVIROTIME: DIGITAL CLOCK
WITH REAL-TIME TEMPERATURE
MONITORING**

Submitted by
Gayathri Kolli
V25HE8K7

Acknowledgement

I would like to express my sincere gratitude to our department faculty, guide, and institution for providing me the opportunity to carry out this mini project titled “**EnviroTime: Digital Clock with Real-Time Temperature Monitoring.**” Their guidance, encouragement, and technical support helped me complete this project successfully.

Table of contents

S.No	Title
1	Introduction
2	Aim
3	Objectives
4	Block Diagram
5	Hardware Requirements
6	Software Requirements
7	Component Description
8	Circuit Diagram
9	Flowchart
10	Program Explanation & Source Code
11	Results and Output
12	Applications
13	Advantages
14	Conclusion
15	Future Scope
16	References

1. Introduction:

Embedded systems play an important role in automation and real-time monitoring applications. This project **EnviroTime**, is designed using the NXP **LPC2148** microcontroller to display accurate time and continuously monitor ambient temperature. The system combines Real-Time Clock functionality, temperature sensing, password-based security, alarm indication, and user interaction through keypad and switches. The measured information is displayed on a 16×2 LCD, making the system suitable for homes, offices, laboratories, and industrial applications.

2. Aim:

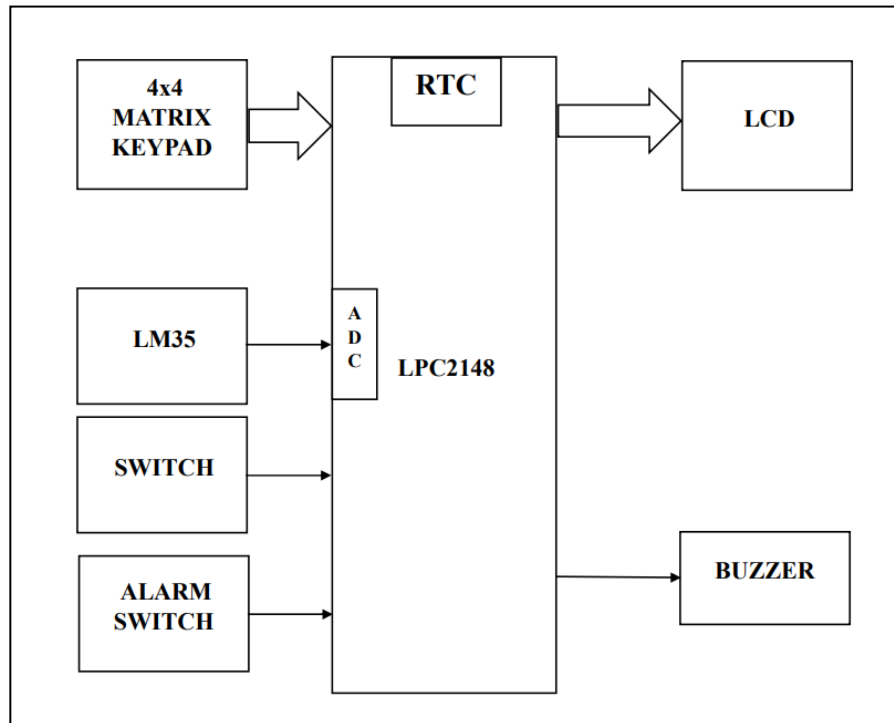
To design and develop a real-time embedded system using the LPC2148 microcontroller that displays accurate digital clock information along with continuous ambient temperature monitoring, while allowing secure user-controlled modification of time and alarm settings through switch-based polling and keypad authentication.

3. Objectives:

- Implement Real-Time Clock (RTC) for accurate display of time, date, day, month, and year. Monitor ambient temperature using LM35 sensor through on-chip ADC
- Display real-time clock and temperature information on a 16×2 LCD

- Provide alarm setting functionality for user-defined time alert
- Interface keypad for secure password-based user input and parameter modification
- Use switch-based polling to enter edit mode for updating RTC and alarm settings
- Generate buzzer alerts for alarm indication and system notifications

4. Block Diagram:



Note: In real hardware, we used buzzer but on Proteus simulation for visibility of output we used LED for alarm indication

5. HARDWARE REQUIREMENTS:

- LPC2148
- 16X2 LCD
- 4X4 MATRIX KEYPAD
- SWITCH
- LM35
- BUZZER
- USB-UART CONVERTER / DB-9 CABLE

6. SOFTWARE REQUIREMENTS:

- EMBEDDED-C PROGRAMMING
- FLASH MAGIC

7. Component description:

7.1 LPC2148 :

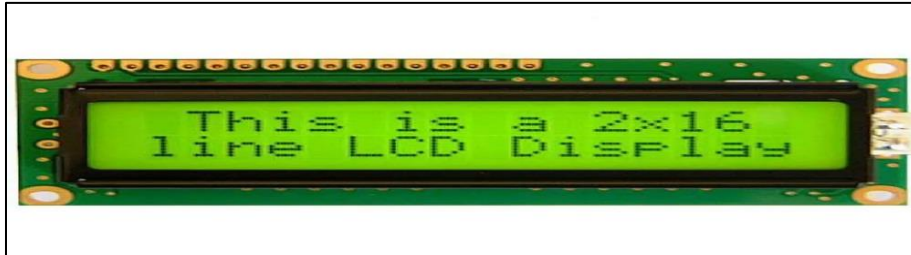
The LPC2148 is a 32-bit ARM7 based microcontroller developed by NXP Semiconductors. It is the main controller used in this project. It controls all peripheral devices such as LCD, keypad, switches, buzzer, and temperature sensor. It contains built-in features like ADC,

RTC, UART, GPIO, timers, and interrupt support, making it suitable for real-time embedded applications.

Important features:

- 32-bit ARM7TDMI-S core
- Operating frequency up to 60 MHz
- Built-in ADC
- Built-in RTC
- UART communication support
- Multiple GPIO pins
- 32 KB RAM, 512 KB Flash memory

7.2 16×2 LCD Display:



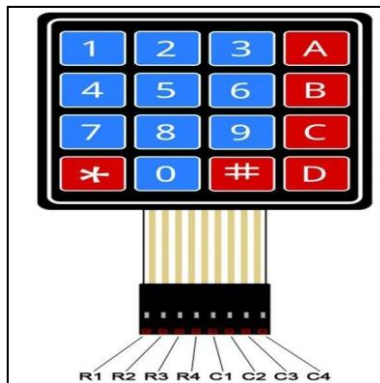
A 16×2 LCD display is used to display system information. It can display 16 characters in 2 rows.

In this project, it displays:

- Current time
- Date and day
- Temperature
- Alarm messages
- Password prompts

The LCD provides a user-friendly interface.

7.3 4×4 Matrix Keypad :



The keypad is used to enter numerical values and passwords. It consists of 16 keys arranged in 4 rows and 4 columns.

Functions:

- Password entry

- Time modification
- Alarm setting
- Password change

7.4 Switch :



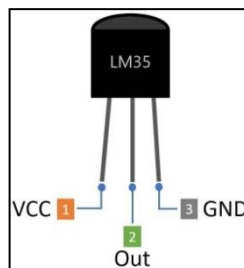
Switches are used for manual control.

Functions:

- Enter edit mode
- Stop alarm

The switches are monitored using polling.

7.5 LM35 :



LM35 is an analog temperature sensor used to measure ambient temperature. It produces an output voltage proportional to temperature in degree Celsius.

Features:

- Calibrated in Celsius
- Low power consumption
- High accuracy
- Easy ADC interfacing

7.6 Buzzer:



The buzzer generates sound alerts.

Functions:

- Alarm indication
- Security alerts
- Wrong password indication

7.7 USB-UART Converter / DB-9 Cable:



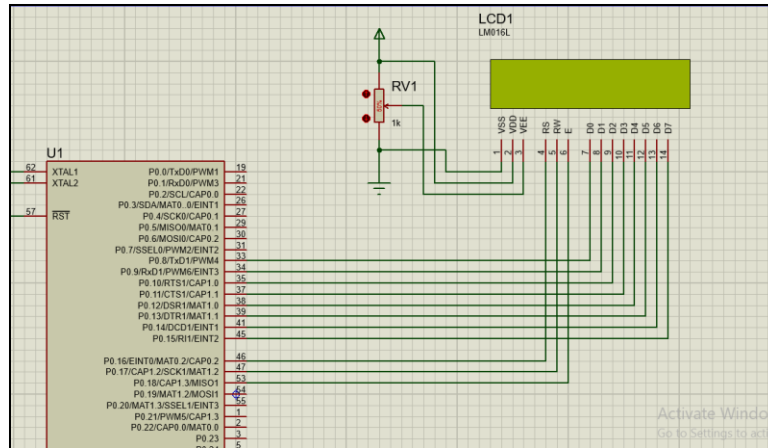
This module is used for serial communication between the computer and the **LPC2148** board.

Functions:

- Program downloading
- Serial communication
- Debugging
- Flash programming

8. Circuit Connections

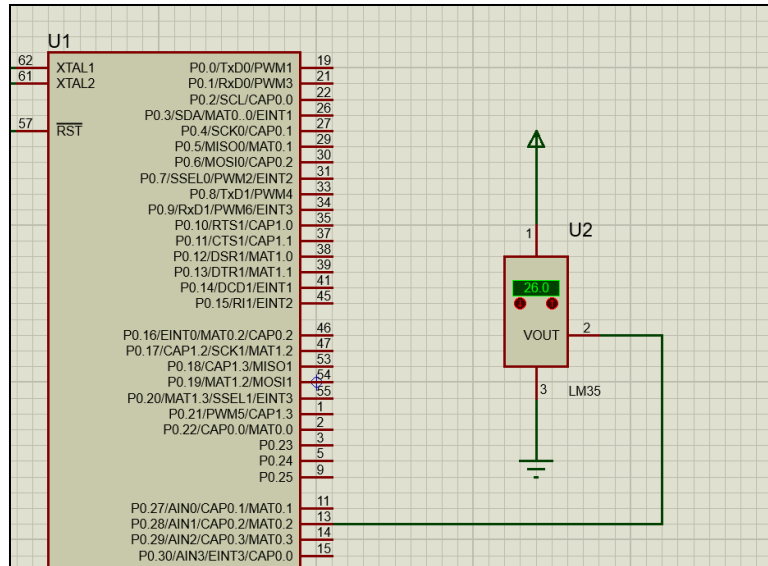
8.1 LPC2148 to 16×2 LCD :



The LCD is interfaced with GPIO pins of the microcontroller in 8-bit mode.

LCD Pin	Signal	Connected To
Pin 1	VSS	GND
Pin 2	VDD	+5V
Pin 3	VEE	Potentiometer (RV1) output
Pin 4	RS	P0.16
Pin 5	RW	P0.17
Pin 6	EN	P0.8
Pin 7	D0	P0.9
Pin 8	D1	P0.10
Pin 9	D2	P0.11
Pin 10	D3	P0.12
Pin 11	D4	P0.13
Pin 12	D5	P0.14
Pin 13	D6	P0.15
Pin 14	D7	P0.18

8.2 LPC2148 to LM35:

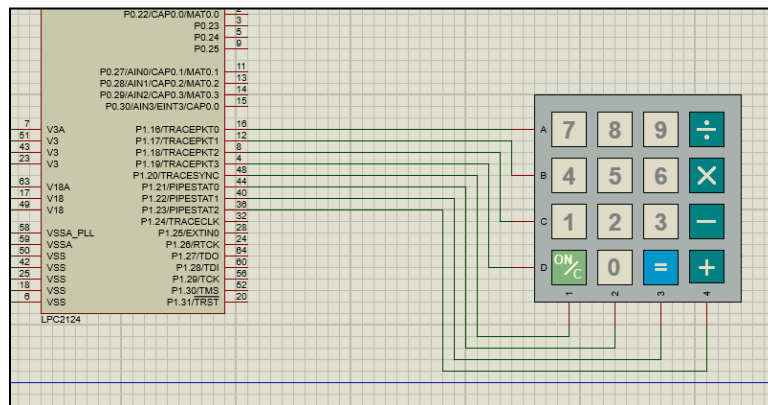


LM35 output is connected to ADC input of the microcontroller.

LM35 Pin	Signal	Connected To
Pin 1	VCC	+5V
Pin 2	Output	P0.28 (AD0.1)
Pin 3	GND	Ground

The analog signal is converted into digital temperature values.

8.3 Keypad Interfacing with LPC2148:

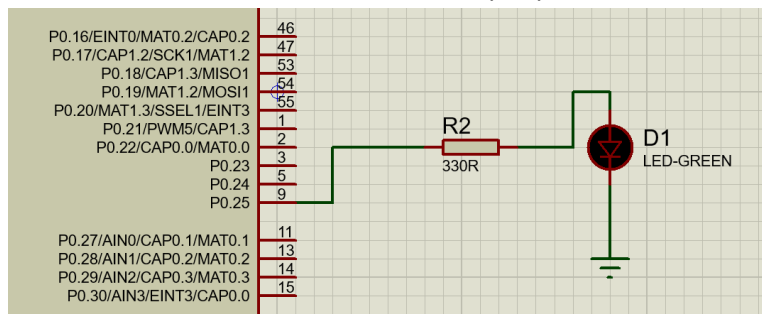


The 4x4 matrix keypad is used for secure user input such as password entry, time modification, alarm setting, and password change. The keypad consists of 4 rows and 4 columns.

Keypad Pin	Signal	Connected To
Pin 1	Row A	P1.16
Pin 2	Row B	P1.17
Pin 3	Row C	P1.18
Pin 4	Row D	P1.19
Pin 5	Column 1	P1.20
Pin 6	Column 2	P1.21
Pin 7	Column 3	P1.22
Pin 8	Column 4	P1.23

8.4 Buzzer Interfacing with LPC2148:

The buzzer is connected to one GPIO output pin.



An Active High LED is used instead of a buzzer in **Proteus** for better visual indication of alarm and security notifications during simulation.

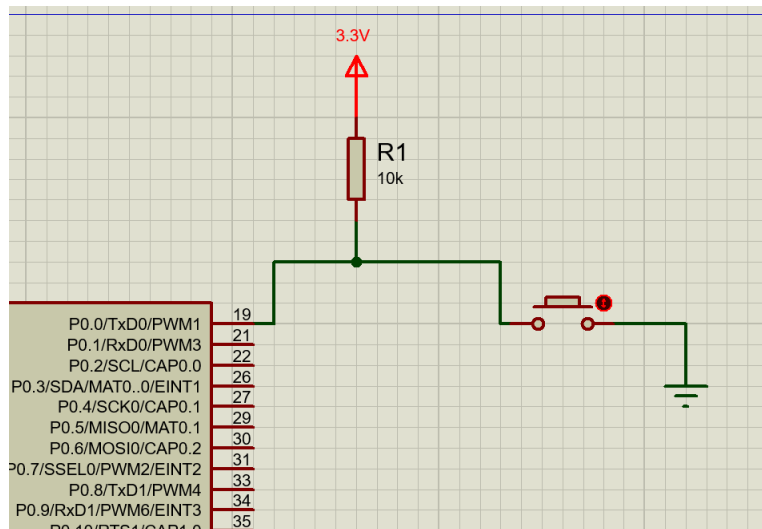
In real hardware buzzer is used.

Component	Signal	Connected To
LED Anode	Positive terminal	P0.25 through 330Ω resistor
LED Cathode	Negative terminal	Ground

8.5 Switch Interfacing with LPC 2148 :

Switches are active low.

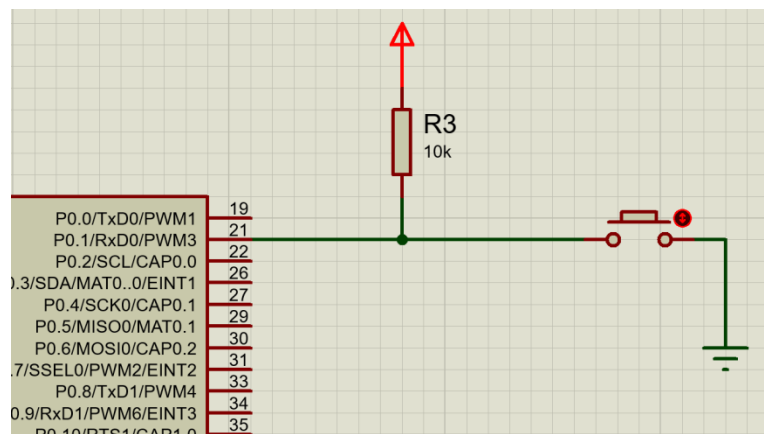
8.5.1 Edit Switch



Component	Signal	Connected To
Switch Terminal 1	Input	P0.0
Switch Terminal 2	Ground	GND
Pull-up Resistor	10k Ω	+3.3V

The edit switch is used to enter secure edit mode. When this switch is pressed, the microcontroller prompts the user to enter the password through the keypad. After successful authentication, the user can modify time, date, alarm settings, and password.

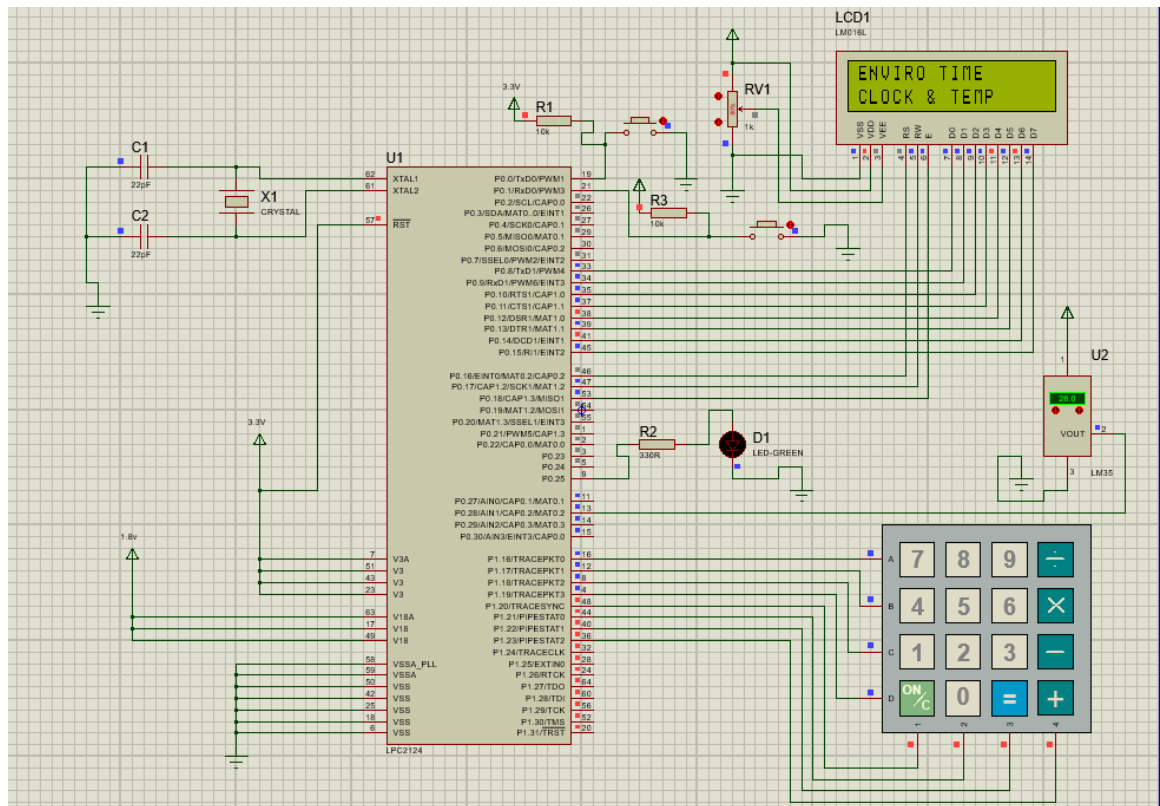
8.5.2 Alarm Stop Switch



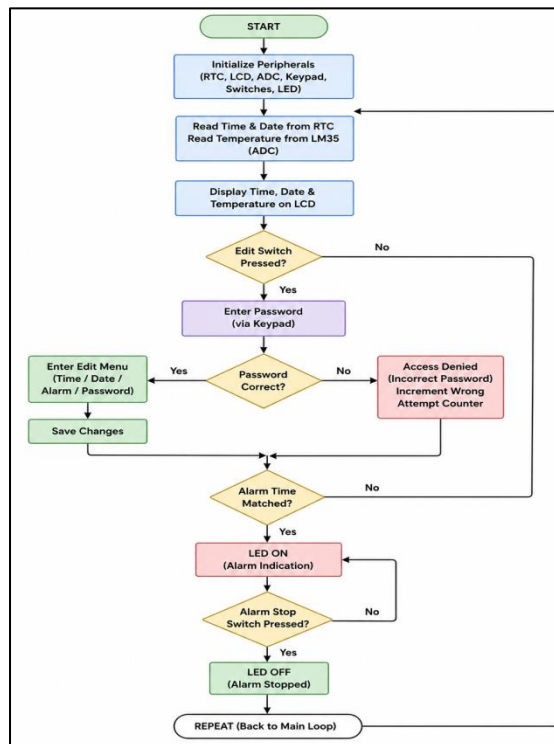
Component	Signal	Connected To
Switch Terminal 1	Input	P0.1
Switch Terminal 2	Ground	GND
Pull-up Resistor	10k Ω	+3.3V

The alarm stop switch is used to stop the alarm indication. When the alarm time matches the current RTC time, the LED glows. Pressing this switch turns OFF the LED and returns the system to normal monitoring mode.

8.6 Full Circuit connections :



9. Flow chart :



10. Program Explanation / Source Code:

10.1 Program Explanation :

- The program is written in **Embedded C** for the LPC2148 microcontroller. It initializes all required peripherals such as LCD, RTC, ADC, keypad, switches, and LED. After initialization, the controller continuously reads the real-time clock values and temperature sensor value, then displays them on the 16×2 LCD.
- The edit switch is checked using polling. When the edit switch is pressed, the system asks the user to enter a password through the 4×4 keypad. If the password is correct, the edit menu is displayed, where the user can modify time, date, alarm time, or password. If the password is incorrect, access is denied.
- The alarm time is continuously compared with the current RTC time. When both match, the LED turns ON as an alarm indication. The LED remains ON until the alarm stop switch is pressed.

10.2 Source Code :

10.2.1 Main.c

```
#include <lpc21xx.h>
#include "type.h"
#include "kpm.h"
#include "lcd.h"
#include "rtc.h"
#include "lcd_defines.h"
#include "kpm_defines.h"
#include "Intial.h"
#include "delay.h"
#include "adc.h"
#include "lm35.h"
#define BUZZER_PIN      25
#define EDIT_SW         0
#define ALARM_STOP_SW  1
extern s8 default_password[5];
u32 h = 12;
u32 mi = 30;
u32 s = 0;
u32 d = 3;
u32 mo = 5;
u32 y = 2026;
u32 da = 1;
u32 alarm_hour = 12;
u32 alarm_min = 31;

u32 wrong_count = 0;
u32 alarm_flag = 0;
int main()
{
    Init_system();
    //P0.25 as GPIO output for buzzer/LED
    PINSEL1 &= ~(3 << 18);
    IODIR0 |= (1 << BUZZER_PIN);
```

```

CmdLCD(CLEAR_LCD);
StrLCD("ENVIRO TIME");
CmdLCD(GOTO_LINE2_POS0);
StrLCD("CLOCK & TEMP");
delay_ms(1500);
CmdLCD(CLEAR_LCD);
while(1)
{
    display_envirotime();
    check_alarm();
    //EDIT SWITCH
    if(((IOPIN0 >> EDIT_SW) & 1) == 0)
    {
        delay_ms(50);
        if(((IOPIN0 >> EDIT_SW) & 1) == 0)
        {
            while(((IOPIN0 >> EDIT_SW) & 1) == 0);
            if(wrong_count >= 3)
            {
                u32 sec_count, beep;
                //buzzer beeps 3 times
                for(beep = 0; beep < 3; beep++)
                {
                    IOSET0 = 1 << BUZZER_PIN;
                    delay_ms(200);
                    IOCLR0 = 1 << BUZZER_PIN;
                    delay_ms(200);
                }
                CmdLCD(CLEAR_LCD);
                delay_ms(5);
                CmdLCD(GOTO_LINE1_POS0);
                StrLCD("SYSTEM LOCKED");

                for(sec_count = 45; sec_count > 0; sec_count--)
                {
                    CmdLCD(GOTO_LINE2_POS0);
                    StrLCD("WAIT: ");
                    CharLCD((sec_count / 10) + '0');
                    CharLCD((sec_count % 10) + '0');
                    StrLCD(" SEC ");
                    delay_ms(1000);
                }
                wrong_count = 0;
                CmdLCD(CLEAR_LCD);
                delay_ms(5);
                continue;
            }
            if(password())
            {
                wrong_count = 0;
                process_edit_menu();
            }
            else

```

```

        {
            wrong_count++;
            CmdLCD(CLEAR_LCD);
            StrLCD("ACCESS DENIED");
            delay_ms(500);
            IOSET0 = 1 << BUZZER_PIN;
            delay_ms(1000);
            IOCLR0 = 1 << BUZZER_PIN;
            CmdLCD(CLEAR_LCD);
            delay_ms(5);
        }
    }
}
//ALARM STOP SWITCH
if(alarm_flag == 1 &&
    (((IOPIN0 >> ALARM_STOP_SW) & 1) == 0))
{
    IOCLR0 = 1 << BUZZER_PIN;
    alarm_flag = 0;
    while(((IOPIN0 >> ALARM_STOP_SW) & 1) == 0);
}
delay_ms(200);
}
}

```

10.2.2 Adc.c

```

#include <lpc21xx.h>
#include "adc.h"
void Init_ADC(void)
{
    PINSEL1|=1<<24;
    ADCR=(1<<1)|
        (4<<8)|
        (1<<21);
}
u32 Read_ADC(void)
{
    ADCR|=1<<24;
    while(((ADDR>>31)&1)==0);
    return (ADDR>>6)&1023;
}

```

10.2.3 Alarm.c

```

#include "Intial.h"
#include <lpc21xx.h>
#include "lcd.h"
#include "rtc.h"
#include "lcd_defines.h"
#include "delay.h"
#include "type.h"
#define BUZZER_PIN 25
extern u32 alarm_hour, alarm_min;
extern u32 alarm_flag;
void alarm_time_setting(void)

```

```

{
    CmdLCD(CLEAR_LCD);
    StrLCD("ALARM HOUR");
    alarm_hour = get_hour_from_user();
    CmdLCD(CLEAR_LCD);
    StrLCD("ALARM MIN");
    alarm_min = get_min_from_user();
    CmdLCD(CLEAR_LCD);
    StrLCD("ALARM SAVED");
    delay_ms(1000);
    CmdLCD(CLEAR_LCD);
}
void check_alarm(void)
{
    s32 h, m, s;
    static u32 alarm_done = 0;
    GetRTCTimeInfo(&h, &m, &s);
    //trigger only once
    if((h == alarm_hour) &&
        (m == alarm_min) &&
        (alarm_done == 0))
    {
        alarm_flag = 1;
        IOSET0 = (1 << BUZZER_PIN);
        alarm_done = 1;
    }
    //reset for next day/alarm
    if((h != alarm_hour) || (m != alarm_min))
    {
        alarm_done = 0;
    }
}

```

10.2.4 Delay.c

```

#include "type.h"
#include "delay.h"
void delay_ms(u32 DLYMS)
{
    for(DLYMS*=12000;DLYMS>0;DLYMS--);
}
void delay_us(u32 DLYUS)
{
    for(DLYUS*=12;DLYUS>0;DLYUS--);
}
void delay_s(u32 DLYS)
{
    for(DLYS*=12000000;DLYS>0;DLYS--);
}

```

10.2.5 Device_settings.c

```

#include "kpm.h"
#include "lcd.h"
#include "delay.h"
#include "type.h"

```

```

#include "lcd_defines.h"
u32 get_number_from_keypad(void)
{
    char buf[3] = {0,0,0};
    char key;
    u32 value = 0;
    int pos = 0;
    CmdLCD(GOTO_LINE2_POS0);
    while(1)
    {
        key = KeyScan();
        delay_ms(120);
        if(key >= '0' && key <= '9')
        {
            if(pos < 2)
            {
                buf[pos] = key;
                CmdLCD(GOTO_LINE2_POS0 + pos);
                CharLCD(key);
                pos++;
            }
        }
        else if(key == 'B')
        {
            if(pos > 0)
            {
                pos--;
                buf[pos] = '\0';
                CmdLCD(GOTO_LINE2_POS0 + pos);
                CharLCD(' ');
                CmdLCD(GOTO_LINE2_POS0 + pos);
            }
        }
        else if(key == 'S')
        {
            if(pos == 1)
            {
                value = buf[0] - '0';
                return value;
            }
            else if(pos == 2)
            {
                value = ((buf[0] - '0') * 10) + (buf[1] - '0');
                return value;
            }
        }
    }
}

u32 get_hour_from_user(void)
{
    u32 h;
    while(1)
    {

```



```

        CmdLCD(CLEAR_LCD);
        delay_ms(300);
        StrLCD("Enter Hour 0-23");
        h = get_number_from_keypad();
        if(h <= 23)
            return h;
        CmdLCD(CLEAR_LCD);
        StrLCD("Invalid Hour!");
        delay_ms(1000);
    }
}
u32 get_min_from_user(void)
{
    u32 m;
    while(1)
    {
        CmdLCD(CLEAR_LCD);
        delay_ms(300);
        StrLCD("Enter Min 0-59");
        m = get_number_from_keypad();
        if(m <= 59)
            return m;
        CmdLCD(CLEAR_LCD);
        StrLCD("Invalid Minute!");
        delay_ms(800);
    }
}
u32 select_time_edit_option(void)
{
    char opt[][16] = {
        "1.HOUR",
        "2.MINUTE",
        "3.HOUR+MIN",
        "4.EXIT"
    };
    u32 s = 0;
    char key;
    while(1)
    {
        CmdLCD(CLEAR_LCD);
        StrLCD(opt[s]);
        delay_ms(200);
        key = KeyScan();
        if(key == 'U')
        {
            if(s == 0)
                s = 3;
            else
                s--;
        }
        else if(key == 'D')
        {
            if(s == 3)

```

```

        s = 0;
    else
        s++;
    }
    else if(key == 'S')
    {
        return s;
    }
}
}

```

10.2.6 Display_rtc.c

```

#include "lcd.h"
#include "lcd_defines.h"
#include "rtc.h"
#include "lm35.h"

void display_envirotime(void)
{
    s32 hour, min, sec;
    s32 date, month, year;
    s32 day;
    u32 temp;

    GetRTCTimeInfo(&hour, &min, &sec);
    GetRTCDateInfo(&date, &month, &year);
    GetRTCDay(&day);

    temp = Read_LM35_Temp();

    //LINE 1: HH:MM:SS T:27C
    CmdLCD(GOTO_LINE1_POS0);

    DisplayRTCTime(hour, min, sec);

    CmdLCD(GOTO_LINE1_POS0 + 9);
    StrLCD("T:");

    if(temp < 10)
    {
        CharLCD('0');
        CharLCD(temp + '0');
    }
    else
    {
        CharLCD((temp / 10) + '0');
        CharLCD((temp % 10) + '0');
    }

    CharLCD(0xDF);
    CharLCD('C');

    //LINE 2: DD/MM/YYYY WED

```

```

    CmdLCD(GOTO_LINE2_POS0);

    DisplayRTCDate(date, month, year);

    CmdLCD(GOTO_LINE2_POS0 + 11);

    DisplayRTCDay(day);
}

```

10.2.7 Edit_rtc_settings.c

```

#include "type.h"
#include "delay.h"
#include "lcd.h"
#include "Intial.h"
#include "rtc.h"
#include "kpm.h"
#include "lcd_defines.h"

extern u32 h,mi,s,d,mo,y,da;
void edit_rtc_settings(void)
{
    u32 choice;

    while(1)
    {
        choice = rtc_edit_menu();
        if(choice == 7)
        {
            while(ColScan()==0);
            delay_ms(150);

            SetRTCTimeInfo(h,mi,s);
            SetRTCDateInfo(da,mo,y);
            SetRTCDay(d);

            CmdLCD(CLEAR_LCD);
            StrLCD("RTC UPDATED");
            delay_ms(600);
            CmdLCD(CLEAR_LCD);

            display_envirotime();
            delay_ms(800);
            return;
        }

        switch(choice)
        {
            case 0: set_rtc_hour(&h);      break;
            case 1: set_rtc_minute(&mi);    break;
            case 2: set_rtc_second(&s);      break;
            case 3: set_rtc_day(&d);         break;
            case 4: set_rtc_date(&da);      break;

```

```

        case 5: set_rtc_month(&mo);    break;
        case 6: set_rtc_year(&y);      break;
    }
}
u32 rtc_edit_menu(void)
{
    char menu[][15]={"1.HOUR","2.Minute","3.Second",
    ,"4.Day","5.Date","6.Month","7.Year","8.Exit"};
    u32 selected = 0;
    char key;

    while(1)
    {
        CmdLCD(CLEAR_LCD);
        StrLCD(menu[selected]);
        delay_ms(200);

        key = KeyScan();

        if(key == 'U')
        {
            if(selected == 0)
                selected = 7;
            else
                selected--;
        }
        else if(key == 'D')
        {
            if(selected == 7)
                selected = 0;
            else
                selected++;
        }
        else if(key == 'S')
        {
            return selected;
        }
    }
}

void set_rtc_hour(u32 *h)
{
    u32 temp;

    while(1)
    {
        CmdLCD(CLEAR_LCD);
        delay_ms(500);
        StrLCD("Enter Hour 0-23");

        temp = get_number_from_keypad();

        if(temp <= 23)

```

```

        {
            *h=temp;
            CmdLCD(CLEAR_LCD);
            delay_ms(2);
            StrLCD("HOUR UPDATED");
            delay_ms(600);
            return;
        }

        CmdLCD(CLEAR_LCD);
        StrLCD("Invalid Hour!");
        delay_ms(600);
    }
}

void set_rtc_minute(u32 *mi)
{
    u32 temp;

    while(1)
    {
        CmdLCD(CLEAR_LCD);
        delay_ms(500);
        StrLCD("Enter Min 0-59");

        temp = get_number_from_keypad();

        if(temp <= 59)
        {
            *mi=temp;
            CmdLCD(CLEAR_LCD);
            delay_ms(2);
            StrLCD("MIN UPDATED");
            delay_ms(600);
            return;
        }

        CmdLCD(CLEAR_LCD);
        StrLCD("Invalid Minute!");
        delay_ms(600);
    }
}

void set_rtc_second(u32 *s)
{
    u32 temp;

    while(1)
    {
        CmdLCD(CLEAR_LCD);
        delay_ms(500);
        StrLCD("Enter Sec 0-59");

        temp = get_number_from_keypad();
    }
}

```

```

        if(temp<= 59)
        {
            *s=temp;
            CmdLCD(CLEAR_LCD);
            delay_ms(2);
            StrLCD("SEC UPDATED");
            delay_ms(600);
            return;
        }

        CmdLCD(CLEAR_LCD);
        StrLCD("Invalid Second!");
        delay_ms(600);
    }
}

void set_rtc_day(u32 *d)
{
    u32 temp;

    while(1)
    {
        CmdLCD(CLEAR_LCD);
        delay_ms(500);
        StrLCD("Day 0-6:");

        temp = get_number_from_keypad();

        if(temp <= 6)
        {
            *d=temp;
            CmdLCD(CLEAR_LCD);
            delay_ms(2);
            StrLCD("DAY UPDATED");
            delay_ms(600);
            return;
        }

        CmdLCD(CLEAR_LCD);
        StrLCD("Invalid Day!");
        delay_ms(600);
    }
}

void set_rtc_year(u32 *y)
{
    char yr[5];
    u32 index;
    u32 temp;
    char key;
    int i;

    while(1)

```

```

{
    CmdLCD(CLEAR_LCD);
    delay_ms(200);
    StrLCD("YEAR 2000-2099");
    index = 0;
    for(i = 0; i < 5; i++)
        yr[i] = 0;
    CmdLCD(GOTO_LINE2_POS0);
    while(1)
    {
        key = KeyScan();
        if(key==0)
            continue;
        if(key >= '0' && key <= '9')
        {
            if(index < 4)
            {
                yr[index] = key;
                CharLCD(key);
                index++;
            }
        }
        else if(key == 'B')
        {
            if(index > 0)
            {
                index--;
                yr[index] = 0;
                CmdLCD(GOTO_LINE2_POS0 + index);
                CharLCD(' ');
                CmdLCD(GOTO_LINE2_POS0 + index);
            }
        }
        else if(key == 'S')
        {
            if(index == 4)
                break;
        }

        delay_ms(150);
    }
    temp = (yr[0]-'0')*1000 +
            (yr[1]-'0')*100 +
            (yr[2]-'0')*10 +
            (yr[3]-'0');
    if(temp >= 2000 && temp <= 2099)
    {
        *y = temp;
        CmdLCD(CLEAR_LCD);
        StrLCD("YEAR UPDATED");
        delay_ms(600);
        CmdLCD(CLEAR_LCD);
        return;
    }
}

```

```

    }
    else
    {
        CmdLCD(CLEAR_LCD);
        StrLCD("Invalid Year!");
        delay_ms(700);
    }
}
}
void set_rtc_month(u32 * mo)
{
    u32 temp;
    while(1)
    {
        CmdLCD(CLEAR_LCD);
        delay_ms(500);
        StrLCD("Month 1-12:");
        temp = get_number_from_keypad();
        if(temp >= 1 && temp <= 12)
        {
            *mo=temp;
            CmdLCD(CLEAR_LCD);
            delay_ms(2);
            StrLCD("MONTH UPDATED");
            delay_ms(500);
            return;
        }
        CmdLCD(CLEAR_LCD);
        StrLCD("Invalid Month!");
        delay_ms(500);
    }
}
u32 days_in_month(u32 month, u32 year)
{
    switch(month)
    {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            // leap year check
            if( (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0) )
                return 29;
            else
                return 28;
        default:
            return 31;
    }
}
void set_rtc_date(u32 *da)
{
    u32 tmp, max_days;

```



```

while(1)
{
    CmdLCD(CLEAR_LCD);
    delay_ms(500);
    StrLCD("Enter Date (1-31)");
    CmdLCD(GOTO_LINE2_POS0);
    tmp = get_number_from_keypad();
    max_days = days_in_month(mo, y);
    if(tmp >= 1 && tmp <= max_days)
    {
        *da = tmp;
        CmdLCD(CLEAR_LCD);
        StrLCD("DATE UPDATED");
        delay_ms(600);
        return;
    }
    CmdLCD(CLEAR_LCD);
    StrLCD("Invalid Date!");
    delay_ms(600);
}
}

```

10.2.8 Iniy_system.c

```

#include <lpc21xx.h>
#include "Intial.h"
#include "lcd.h"
#include "rtc.h"
#include "kpm.h"
#include "adc.h"
#define BUZZER_PIN    25
#define EDIT_SW       0
#define ALARM_STOP_SW 1
extern u32 h,mi,s,d,mo,y,da;
void Init_system(void)
{
    PINSEL0 = 0;
    PINSEL1 = 0;
    Init_LCD();
    Init_RTC();
    Init_kpm();
    Init_ADC();

    // GPIO DIRECTIONS
    IODIR0 |= 1 << BUZZER_PIN;    // P0.19 as output
    IOCLR0 = 1 << BUZZER_PIN;    // buzzer initially OFF
    IODIR0 &= ~(1 << EDIT_SW);    // edit switch input
    IODIR0 &= ~(1 << ALARM_STOP_SW); // alarm stop switch input
    SetRTCTimeInfo(h,mi,s);
    SetRTCDateInfo(da,mo,y);
    SetRTCDay(d);
}

```

10.2.9 Kpm.c

```
#include "kpm_defines.h"
#include "type.h"
#include <lpc21xx.h>
#include "delay.h"
u32 kpmLUT[4][4]={
    {'7','8','9','U'},
    {'4','5','6','D'},
    {'1','2','3','C'},
    {'B','0','S','A'}
};
void Init_kpm(void)
{
    IODIR1 |= 15 << ROW0;    // rows output
    IODIR1 &= ~(15 << COL0); // cols input
    IOCLR1 = 15 << ROW0;    // all rows low
}
u32 ColScan(void)
{
    if(((IOPIN1 >> COL0) & 15) < 15)
    {
        return 0;    // key pressed
    }
    else
    {
        return 1;    // no key pressed
    }
}
u32 RowCheck(void)
{
    u32 r;
    for(r=0; r<4; r++)
    {
        IOSET1 = 15 << ROW0;    // all rows high
        IOCLR1 = 1 << (ROW0 + r); // selected row low
        delay_ms(1);
        if(ColScan() == 0)
            break;
    }
    IOCLR1 = 15 << ROW0;    // rows back low
    return r;
}
u32 ColCheck(void)
{
    u32 c;
    for(c=0; c<4; c++)
    {
        if(((IOPIN1 >> (COL0+c)) & 1) == 0)
            break;
    }
    return c;
}
u32 KeyScan(void)
```

```

{
    u32 r,c,key;
    Init_kpm();
    while(ColScan());          // wait for key press
    delay_ms(20);
    r = RowCheck();
    c = ColCheck();
    key = kpmLUT[r][c];
    while(ColScan() == 0);    // wait for release
    delay_ms(20);
    return key;
}

```

10.2.10 LCD.c

```

#include <lpc21xx.h>
#include "type.h"
#include "lcd_defines.h"
#include "lcd.h"
#include "delay.h"
void WriteLCD(u8 byte)
{
    /***** RW = 0 *****/
    IOCLR0 = 1 << LCD_RW;
    /***** CLEAR LCD DATA PINS P0.8-P0.15 *****/
    IOCLR0 = 0xFF << LCD_DATA;
    /***** WRITE DATA TO P0.8-P0.15 *****/
    IOSET0 = byte << LCD_DATA;
    /***** ENABLE PULSE *****/
    IOSET0 = 1 << LCD_EN;
    delay_us(1);
    IOCLR0 = 1 << LCD_EN;
    delay_ms(2);
}
void CmdLCD(u8 CmdByte)
{
    /***** RS = 0 FOR COMMAND *****/
    IOCLR0 = 1 << LCD_RS;
    WriteLCD(CmdByte);
}
void Init_LCD(void)
{
    /***** LCD DATA + CONTROL PINS AS OUTPUT *****/
    IODIR0 |= 0xFF << LCD_DATA;          // P0.8 to P0.15
    IODIR0 |= 1 << LCD_RS;
    IODIR0 |= 1 << LCD_RW;
    IODIR0 |= 1 << LCD_EN;
    delay_ms(15);
    CmdLCD(MODE_8BIT_1LINE);
    delay_us(4100);
    CmdLCD(MODE_8BIT_1LINE);
    delay_us(100);

    CmdLCD(MODE_8BIT_1LINE);
}

```

```

    CmdLCD(MODE_8BIT_2LINE);
    CmdLCD(DISP_ON_CUR_OFF);
    CmdLCD(CLEAR_LCD);
    CmdLCD(SHIFT_CUR_RIGHT);
}

void CharLCD(u8 asciiVal)
{
    /***** RS = 1 FOR DATA *****/
    IOSET0 = 1 << LCD_RS;
    WriteLCD(asciiVal);
}

void StrLCD(s8 *p)
{
    while(*p)
    {
        CharLCD(*p++);
    }
}

void U32LCD(u32 n)
{
    u8 digit[10];
    s32 i = 0;
    if(n == 0)
    {
        CharLCD('0');
    }
    else
    {
        while(n > 0)
        {
            digit[i++] = (n % 10) + '0';
            n = n / 10;
        }

        for(--i; i >= 0; i--)
        {
            CharLCD(digit[i]);
        }
    }
}

void S32LCD(s32 n)
{
    if(n < 0)
    {
        CharLCD('-');
        n = -n;
    }

    U32LCD(n);
}

```

```

}

void F32LCD(f32 fn,u32 nop)
{
    u32 n,i;
    if(fn < 0.0)
    {
        CharLCD('-');
        fn = -fn;
    }
    n = fn;
    U32LCD(n);
    CharLCD('.');
    for(i=0; i<nop; i++)
    {
        fn = (fn - n) * 10;
        n = fn;
        CharLCD(n + '0');
    }
}

void clear_line2(void)
{
    CmdLCD(GOTO_LINE2_POS0);

    StrLCD("                ");

    CmdLCD(GOTO_LINE2_POS0);
}

```

10.2.11 LM35.C

```

#include "type.h"
#include "adc.h"
u32 Read_LM35_Temp(void)
{
    u32 adc_val;
    f32 voltage;
    u32 temp;
    adc_val = Read_ADC();
    voltage = (adc_val * 3.3) / 1023.0;
    temp = voltage * 100;
    return temp;
}

```

10.2.12 Open_edit_menu.c

```

#include <string.h>
#include "lcd.h"
#include "type.h"
#include "lcd_defines.h"
#include "delay.h"
#include "kpm.h"

char menu[4][16]={
    "1.EDIT RTC",
    "2.SET ALARM",

```

```

        "3.UPDATE PIN",
        "4.EXIT"
    };
    u32 open_edit_menu(void)
    {
        u32 index=0;
        char key;
        while(1)
        {
            CmdLCD(CLEAR_LCD);
            StrLCD(menu[index]);
            key=KeyScan();
            //delay_ms(200);
            if(key=='U')
            {
                if(index==0)
                    index=3;
                else
                    index--;
            }
            else if(key=='D')
            {
                if(index==3)
                    index=0;
                else
                    index++;
            }
            else if(key=='S')
            {
                return index;
            }
        }
    }
}

```

10.2.13 Password.c

```

#include "lcd.h"
#include "type.h"
#include "lcd_defines.h"
#include "kpm.h"
#include "delay.h"
#include<string.h>

#define PASSWORD_LENGTH 4
s8 default_password[PASSWORD_LENGTH+1]="1234";

u32 password(void)
{
    s8 entered[PASSWORD_LENGTH+1],key;
    u32 i=0;

    CmdLCD(CLEAR_LCD);
    delay_ms(2);
    CmdLCD(GOTO_LINE1_POS0);
}

```

```

StrLCD("Enter Password:");
delay_ms(100);
clear_line2();
CmdLCD(GOTO_LINE2_POS0);
while(i < PASSWORD_LENGTH)
{
    key = KeyScan();
    if(key == 'B')
    {
        if(i>0)
        {
            i--; // reduce count
            entered[i]='\0'; // remove last char
            // move cursor back 1 position
            CmdLCD(GOTO_LINE2_POS0+i);
            CharLCD(' '); // overwrite previous * with space
            CmdLCD(GOTO_LINE2_POS0+i);
        }
    }
    else
    {
        entered[i] = key;
        CharLCD(entered[i]);
        delay_ms(150);
        CmdLCD(GOTO_LINE2_POS0+i);
        CharLCD('*');
        i++;
    }
    delay_ms(300);
}
entered[i]='\0';
if(strcmp(entered,default_password)==0)
{
    CmdLCD(CLEAR_LCD);
    delay_ms(2);
    StrLCD("ACCEPTED");
    delay_ms(1000);
    CmdLCD(CLEAR_LCD);
    delay_ms(2);
    return 1;
}
else
{
    CmdLCD(CLEAR_LCD);
    delay_ms(2);
    StrLCD("Invalid password");
    delay_ms(1000);
    CmdLCD(CLEAR_LCD);
    delay_ms(2);
    return 0;
}
}

```

```

#include "Intial.h"
extern s8 default_password[5];
void process_edit_menu(void)
{
    u32 select;
    while(1)
    {
        select=open_edit_menu();
        if(select==3)
            return;
        switch(select)
        {
            case 0:
                edit_rtc_settings();
                break;
            case 1:
                alarm_time_setting();
                break;
            case 2:
                update_password(default_password);
                break;
        }
    }
}

```

10.2.15 Rtc.c

```

#include <lpc21xx.h>
#include "lcd.h"
#include "lcd_defines.h"
#include "type.h"

#define FOSC 12000000
#define CCLK (5 * FOSC)
#define PCLK (CCLK / 4)

#define PREINT_VAL ((PCLK / 32768) - 1)
#define PREFRAC_VAL (PCLK - ((PREINT_VAL + 1) * 32768))

#define RTC_ENABLE (1 << 0)
#define RTC_RESET (1 << 1)
#define RTC_CLKSRC (1 << 4)

s8 week[][4] = {"SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT"};

void Init_RTC(void)
{
    PREINT = PREINT_VAL;
    PREFRAC = PREFRAC_VAL;
    CCR = RTC_RESET;    // reset RTC
    CCR = 0x00;         // release reset
    CCR = RTC_ENABLE;   // start RTC using PCLK
}

```



```

void SetRTCTimeInfo(u32 hour, u32 min, u32 sec)
{
    HOUR = hour;
    MIN = min;
    SEC = sec;
}
void SetRTCDateInfo(u32 date, u32 month, u32 year)
{
    DOM = date;
    MONTH = month;
    YEAR = year;
}
void SetRTCDay(u32 day)
{
    DOW = day;
}
void GetRTCTimeInfo(s32 *hour, s32 *minute, s32 *second)
{
    *hour = HOUR;
    *minute = MIN;
    *second = SEC;
}
void GetRTCDateInfo(s32 *date, s32 *month, s32 *year)
{
    *date = DOM;
    *month = MONTH;
    *year = YEAR;
}
void GetRTCDay(s32 *day)
{
    *day = DOW;
}
void DisplayRTCTime(u32 hour, u32 minute, u32 second)
{
    CharLCD((hour / 10) + '0');
    CharLCD((hour % 10) + '0');
    CharLCD(':');
    CharLCD((minute / 10) + '0');
    CharLCD((minute % 10) + '0');
    CharLCD(':');
    CharLCD((second / 10) + '0');
    CharLCD((second % 10) + '0');
}

void DisplayRTCDate(u32 date, u32 month, u32 year)
{
    CharLCD((date / 10) + '0');
    CharLCD((date % 10) + '0');
    CharLCD('/');
    CharLCD((month / 10) + '0');
    CharLCD((month % 10) + '0');
    CharLCD('/');
}

```

```

        U32LCD(year);
    }
    void DisplayRTCDay(u32 dow)
    {
        if(dow <= 6)
        {
            StrLCD(week[dow]);
        }
        else
        {
            StrLCD("ERR");
        }
    }
}

```

10.2.16 Update_password.c

```

#include "lcd.h"
#include "type.h"
#include "lcd_defines.h"
#include "kpm.h"
#include "delay.h"
#include <string.h>
#include "Intial.h"

#define PASSWORD_LENGTH 4
extern s8 default_password[PASSWORD_LENGTH+1];

void update_password(s8 *saved_pass)
{
    s8 old_entered[5];
    s8 new_password[5];
    s8 confirm_password[5];
    //ASK OLD PASSWORD
    CmdLCD(CLEAR_LCD);
    delay_ms(2);
    StrLCD("ENTER OLD PASSWORD");
    delay_ms(200);
    CmdLCD(GOTO_LINE2_POS0);
    get_password_from_user(old_entered);
    if(strcmp(old_entered, saved_pass) != 0)
    {
        CmdLCD(CLEAR_LCD);
        delay_ms(2);
        StrLCD("WRONG PASSWORD");
        delay_ms(800);
        return;    // exit without update
    }
    //ENTER NEW PASSWORD
    CmdLCD(CLEAR_LCD);
    delay_ms(2);
    StrLCD("ENTER NEW PASSWORD");
    CmdLCD(GOTO_LINE2_POS0);
    get_password_from_user(new_password);
    //CONFIRM PASSWORD

```

```

CmdLCD(CLEAR_LCD);
delay_ms(2);
StrLCD("CONFIRM NEW PASSWORD");
CmdLCD(GOTO_LINE2_POS0);
get_password_from_user(confirm_password);
if(strcmp(new_password, confirm_password) == 0)
{
    strcpy(default_password, new_password);    // update global password
    CmdLCD(CLEAR_LCD);
    StrLCD("PASSWORD UPDATED");
    delay_ms(800);
    CmdLCD(0x01);
}
else
{
    CmdLCD(CLEAR_LCD);
    StrLCD("PASSWORD MISMATCH");
    delay_ms(800);
}
}
void get_password_from_user(s8 *buffer)
{
    s8 key;
    u32 i = 0;

    CmdLCD(GOTO_LINE2_POS0);
    while(i < PASSWORD_LENGTH)
    {
        key = KeyScan();
        if(key >= '0' && key <= '9')
        {
            buffer[i] = key;
            CharLCD('*');
            i++;
        }
        else if(key == 'B')
        {
            if(i > 0)
            {
                i--;
                buffer[i] = '\0';
                CmdLCD(GOTO_LINE2_POS0 + i);
                CharLCD(' ');
                CmdLCD(GOTO_LINE2_POS0 + i);
            }
        }

        delay_ms(200);
    }
    buffer[i] = '\0';
}

```

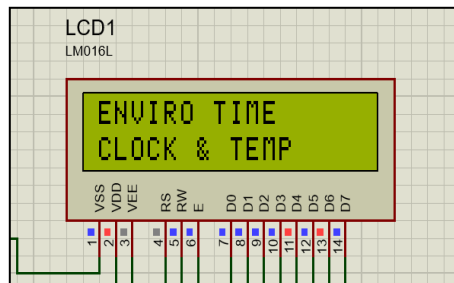
11. Results and Output :

The designed system was successfully implemented using the **LPC2148** microcontroller. The system successfully displayed real-time clock information and ambient temperature on the 16x2 LCD. Password-based secure editing of time and alarm settings was successfully implemented using the 4x4 matrix keypad. The alarm indication was verified using an LED in **Proteus** simulation. The system performed according to the expected design requirements.

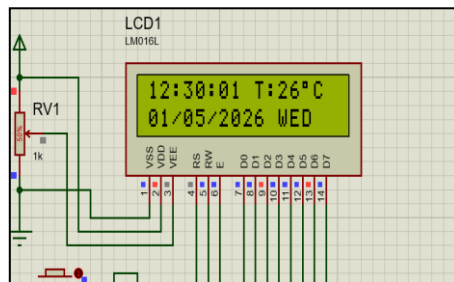
Output Observations

- RTC displayed time correctly
- Temperature displayed correctly from **LM35**
- Password validation worked successfully
- Edit mode functioned properly
- Alarm indication through LED worked successfully
- Alarm stop switch worked correctly

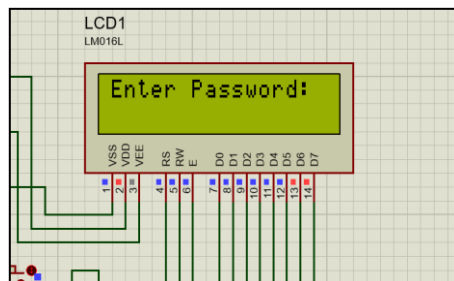
11.1 System Initialization Screen



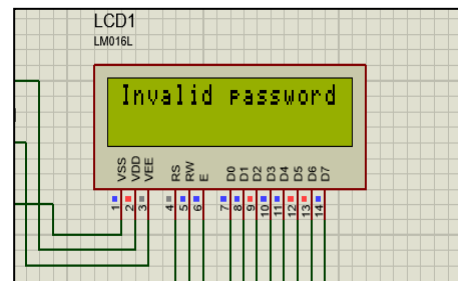
11.2 Real-Time Clock and Temperature Display



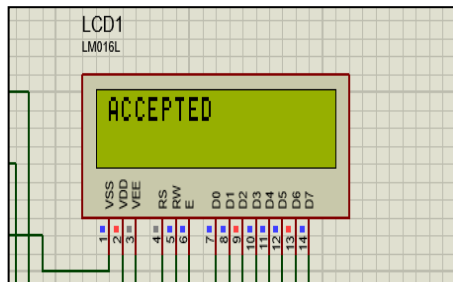
11.3 Password Authentication Screen



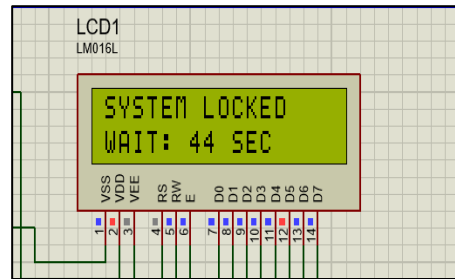
Invalid Password Indication



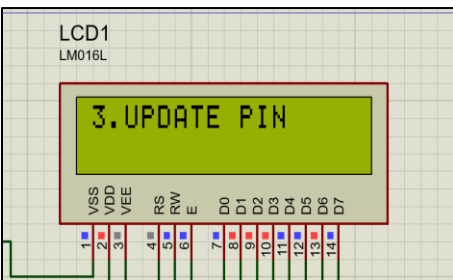
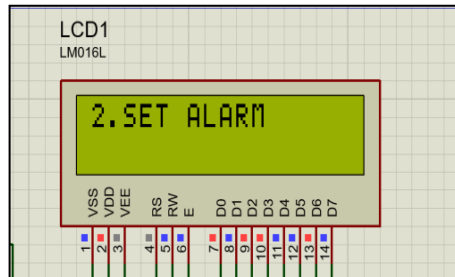
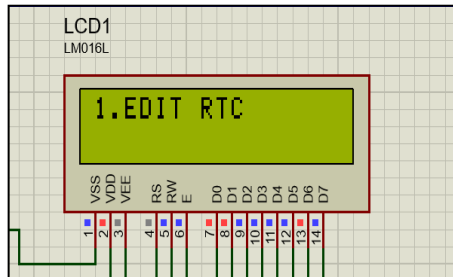
Password correct screen



System locked

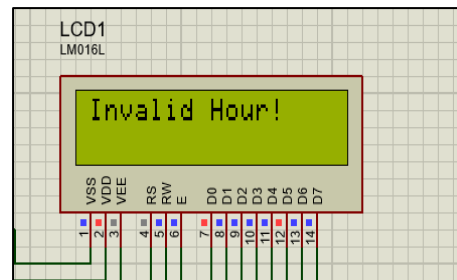
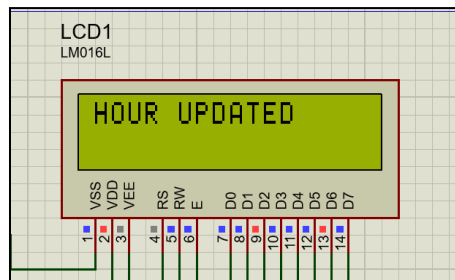
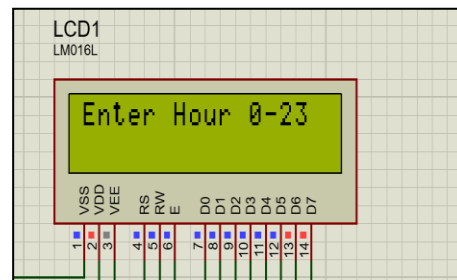
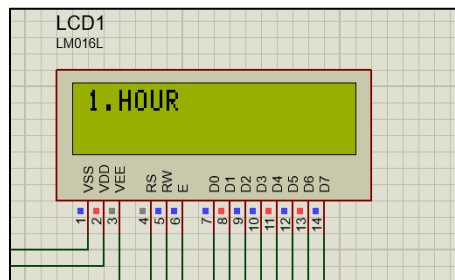


11.4 Main Edit Menu

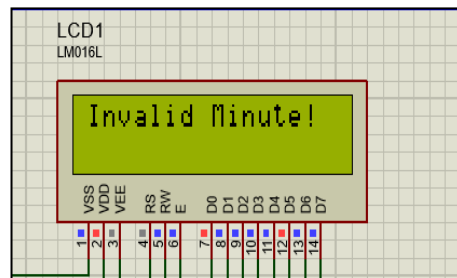
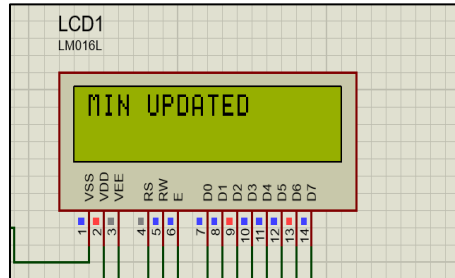
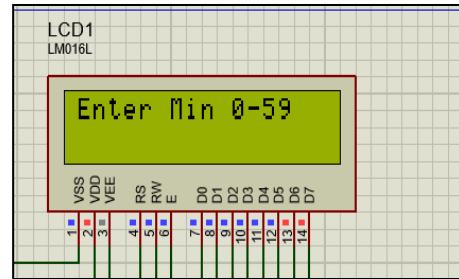
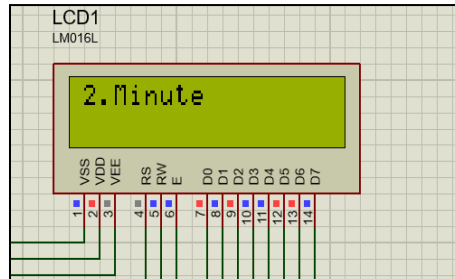


11.5 EDIT RTC

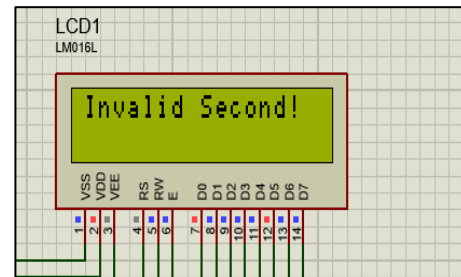
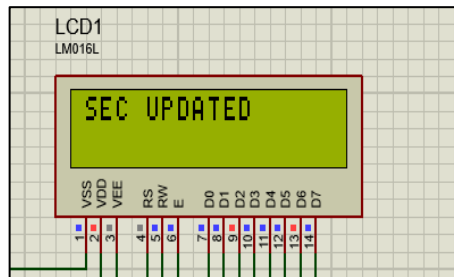
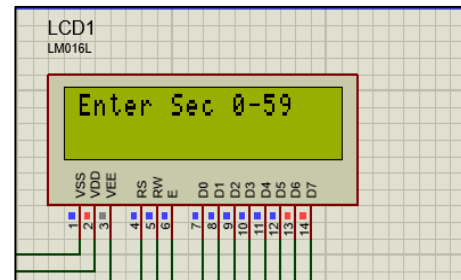
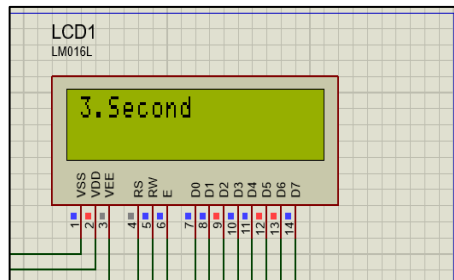
11.5.1 HOUR



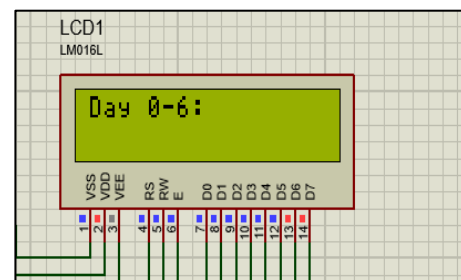
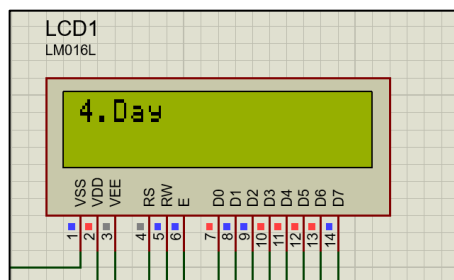
11.5.2 MINUTE

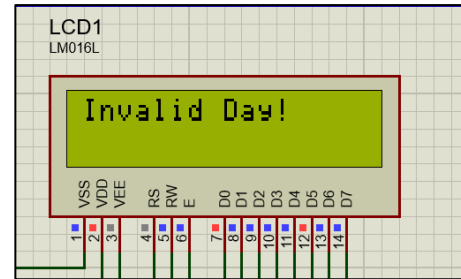
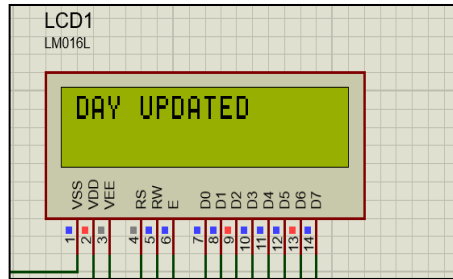


11.5.3 SECOND

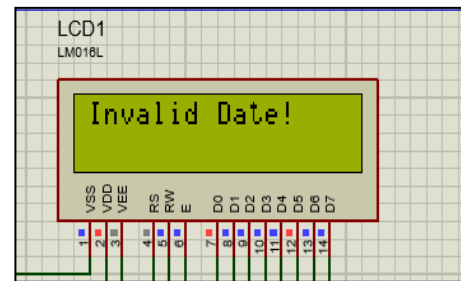
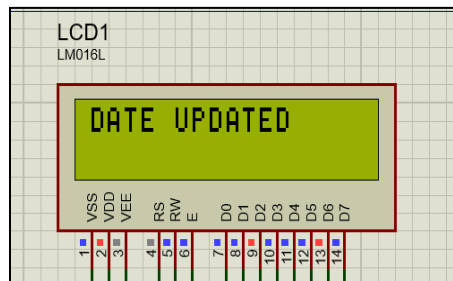
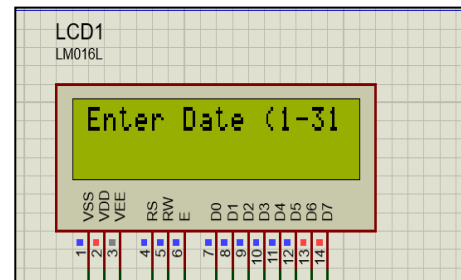
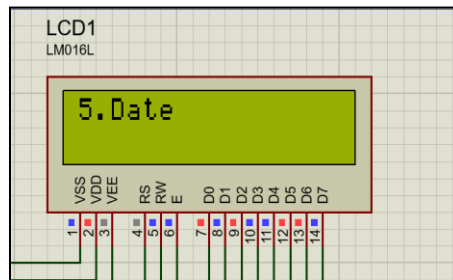


11.5.4 DAY

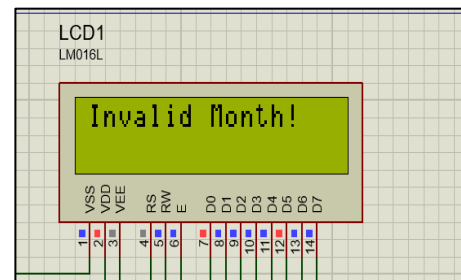
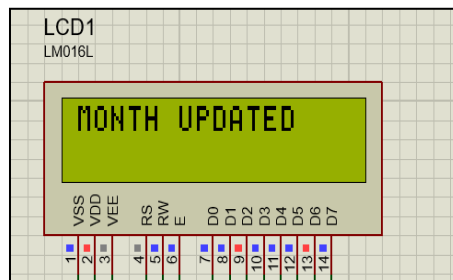
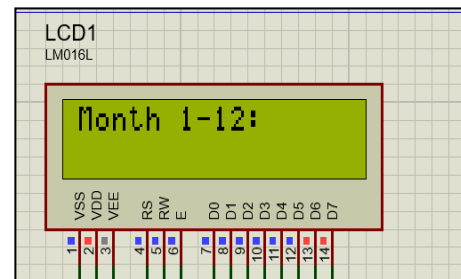
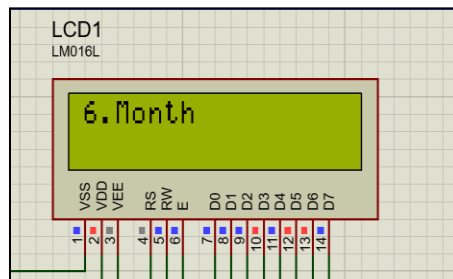




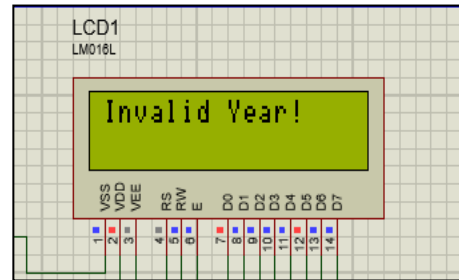
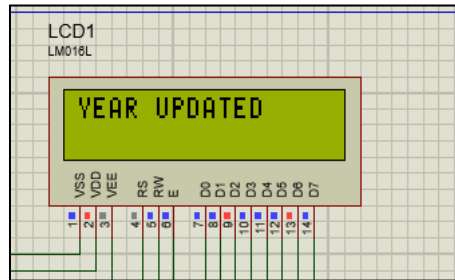
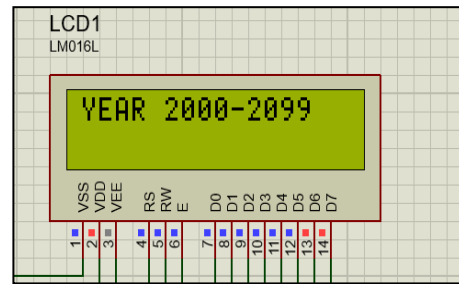
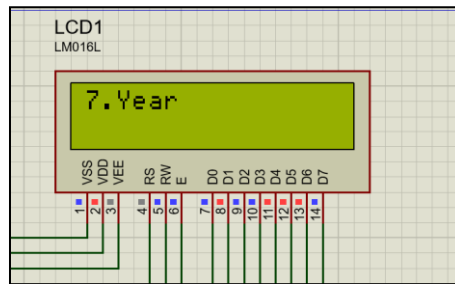
11.5.5 DATE



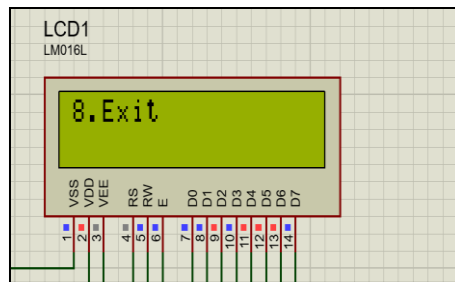
11.5.6 MONTH



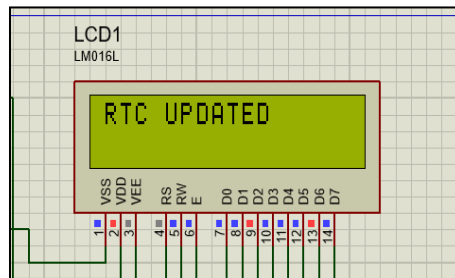
11.5.7 YEAR



11.5.8 EXIT

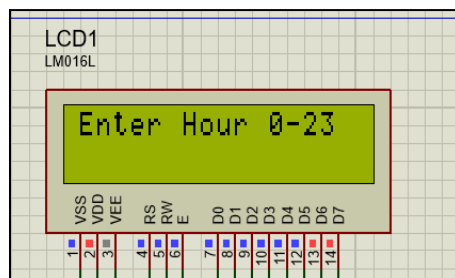


11.5.9 RTC UPDATED

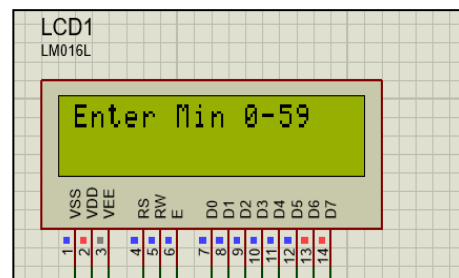


11.6 Alarm setting

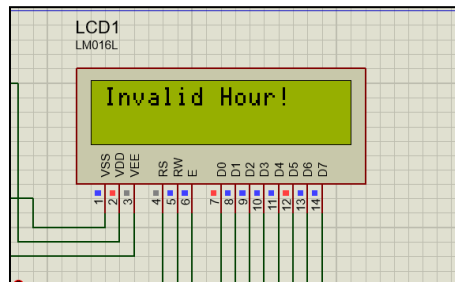
11.6.1 Set hour screen



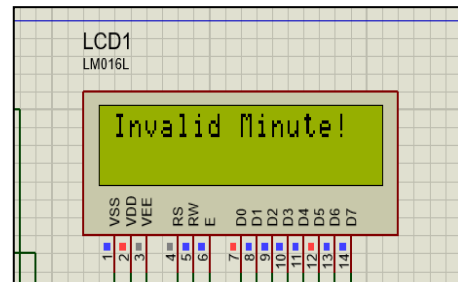
Set min screen



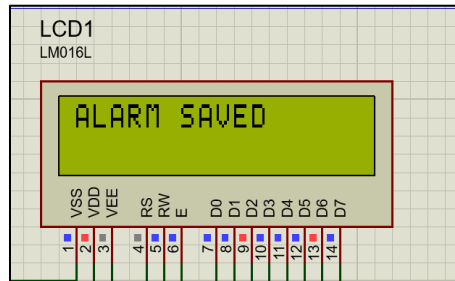
11.6.2 Invalid hour



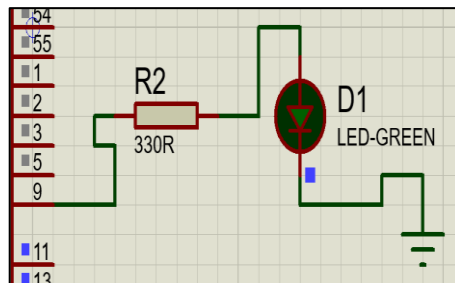
Invalid minute



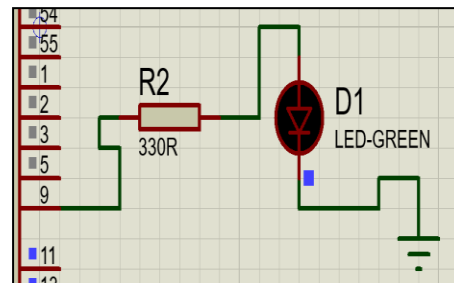
11.6.3 Alarm saved



11.7 Alarm Indication

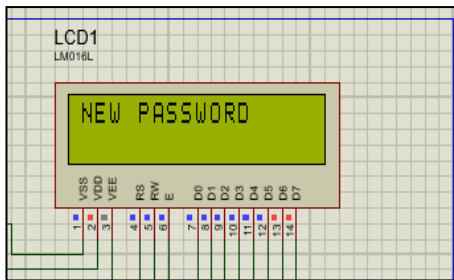
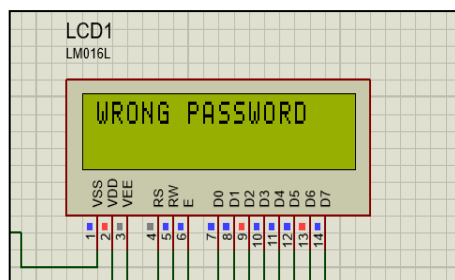
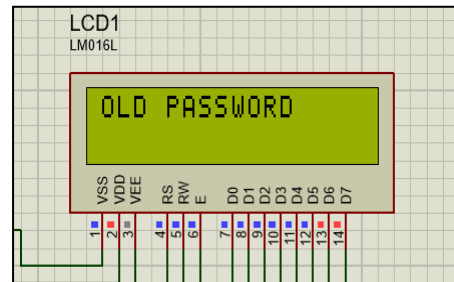
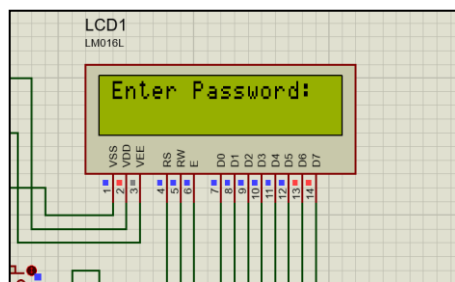


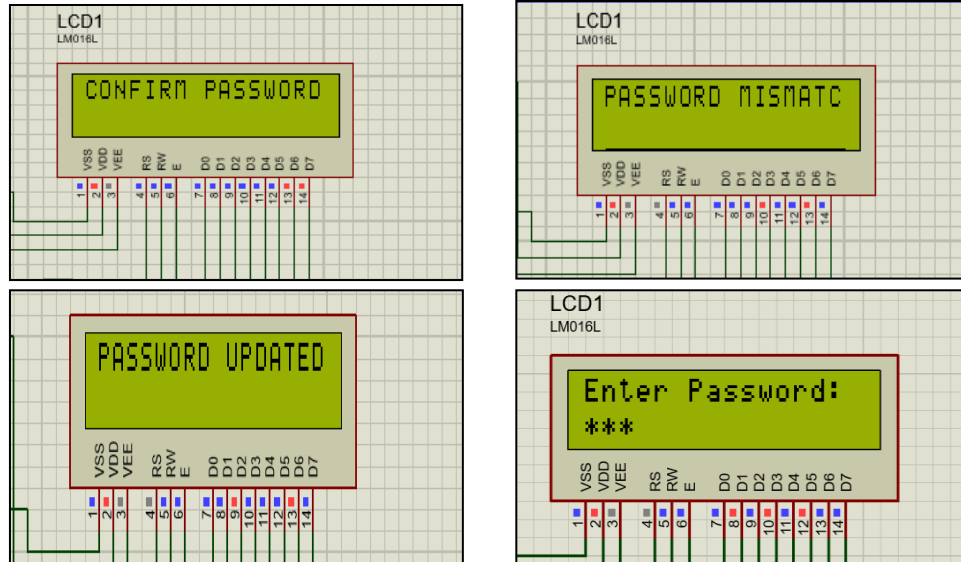
BUZZER ON



Buzzer OFF

11.8 UPDATE PIN





12. Applications:

The applications of this project are:

1. Used in **homes** for displaying time and room temperature.
2. Used in **offices** for real-time clock and temperature monitoring.
3. Used in **laboratories** where temperature monitoring is important.
4. Used in **hospitals** to monitor room temperature and time.
5. Used in **industrial areas** for basic environmental monitoring.
6. Used in **schools and colleges** as a digital clock with alarm and security feature.
7. Can be used as a basic model for **smart home automation systems**.

13. Advantages :

The advantages of this project are:

1. Provides accurate real-time clock display using the built-in RTC of the **LPC2148**.
2. Continuously monitors ambient temperature using the **LM35** sensor.
3. Provides password-based security for editing system parameters.
4. Supports user-friendly interaction through 4x4 matrix keypad and LCD display.
5. Provides alarm indication through LED for easy monitoring.
6. Compact and low-power embedded system.
7. Easy to implement, test, and expand for real-time applications.
8. Suitable for both simulation and real hardware implementation.
9. Improves reliability and security in monitoring applications.

14. Conclusion :

The **EnviroTime** project was successfully designed and implemented using the **LPC2148** microcontroller. The system successfully displayed real-time clock information and ambient temperature on the 16x2 LCD using the **LM35**. Password-based secure access, time and alarm

editing, and alarm indication through LED were implemented successfully. The project achieved all the intended objectives and demonstrated the effective integration of embedded hardware and software for real-time monitoring and security applications.

15. Future Scope:

1. Wireless communication modules such as **ESP8266** or **Bluetooth** can be integrated for remote monitoring and wireless data transmission.
2. Real-time clock and temperature data can be stored in non-volatile memory devices such as **EEPROM** or **Flash Memory** for data logging and analysis.
3. Additional sensors such as humidity sensors, gas sensors, or motion sensors can be interfaced to enhance environmental monitoring capabilities.
4. The system can be integrated with smart home and industrial automation platforms for intelligent monitoring applications.
5. Implementation of temperature history recording for monitoring and analyzing environmental changes over time.
6. Cloud-based monitoring and notification features can be implemented for IoT applications.

16. References:

1. LPC2148 User Manual, NXP Semiconductors.
2. LM35 Temperature Sensor Datasheet, Texas Instruments.
3. Keil μ Vision 4 Software Documentation.
4. Proteus Design Suite User Manual.
5. Embedded C Programming reference materials and laboratory manuals.
6. ARM7 Microcontroller interfacing and embedded systems textbooks.